Action-Effect: An Empirical Protocol for Interchanging Effect Descriptions Without Conflating Evidence and Permission

Ond Holdings Inc.
Toronto, Ontario, Canada

Working draft
First drafted 18 October 2026; revised 19 October 2026
America/Toronto, EDT, UTC-04:00

Working paper for public review

This standalone edition accompanies `manuscript.md`. Retrieval and run times use UTC. America/Toronto is the editorial timezone, not execution geolocation. The Spec release and this paper have separate publication dates.

Abstract

Tool interfaces describe calls, while authorization concerns consequences under particular implementations, arguments, principals, and environments. Existing MCP annotations, rich authorization requests, decision APIs, and policy engines already provide important parts of this contract. We propose a bounded empirical protocol for testing whether an implementation- and context-qualified effect interchange preserves supported meaning across integrations at a justified cost. The protocol separates conformance, native-input mapping, authorization on independently specified reference effects, integrated admission, observed task outcomes, and complete cost. A bounded GitHub issue-comment pilot was executed using MCP and SDK lanes, with a complete structured adapter and the same real OPA policy as the main baseline. Deliberately lossy mapping is an ablation, not a competitive baseline. Mapping errors masked by correct-looking denials, semantic unknowns, processing failures, legitimate work withheld, and ambiguous delivery remain separate outcomes. Both complete strategies matched all 108 native lane instances from 55 authored cases in 22 intent groups, and all 52 eligible decisions. A controlled live run matched 32 planned outcome checks and created 12 comments, including four recovered after deliberately discarded acknowledgments. The protocol's accepted-input offline median cost was 3.97% higher. These same-project results support bounded feasibility, not added safety, performance superiority, independent validation, production readiness, or a new subset-inclusion theorem.

1 Problem and Proposed Contribution

A tool name does not identify every consequence, and a worrying string does not establish a harmful occurrence. An interchange must preserve coupled operation, resource, destination, data, quantity, applicability, and uncertainty without confusing a declaration with authority. This is not an empty space: MCP describes annotations as hints [S1]; AuthZEN supplies authorization exchange [S2]; OAuth RAR already supports detailed permission objects [S3]; and OPA/Cedar evaluate structured facts [S4, S5, S6, S7].

The proposed contribution is a testable contract plus a factorized experiment and failure/custody procedure. Its value would be evidence that different integrations preserve the same supported semantics, expose the same limits, and permit useful controlled work at an acceptable cost. More fields, another serialization, or agreement between same-project implementations is not enough. A complete baseline may be equally correct or cheaper. Such a finding must be retained, not reframed as added safety.

2 Semantic and Trust Model

An effect is a coupled tuple of category, operation, resource, destination, data class, quantities, and modality. A description adds the effective profile, mapping revision, invocation/implementation/context bindings, evidence, coverage, and explicit unresolved facts. It does not redefine the current grant or the execution occurrence. Known, none, and unknown remain distinct. Fields from different alternatives cannot be recombined into an unasserted permission.

2.1 Evidence Is Not Permission

Description processing establishes supported interpretation; a checker can establish only the particular claim covered by its evidence and assumptions. Authorization uses separately authenticated facts and current policy. A final gate controls dispatch, while a provider observation covers an actual occurrence. None of these states implies all the others. Description acceptance remains `authority: not_evaluated`.

Missing required evidence withholds the affected execution but is not a correctly recognized semantic block. An applicable trusted denial survives unrelated uncertainty. Human review can authorize a deliberately limited scope; it cannot make an invalid proof valid. Acceptance of a partial direct-effect profile for one owned test resource cannot silently downgrade another policy's requirement for complete descendant coverage.

2.2 Ordinary Obligations, Not a New Theorem

If a sound checker establishes $E \subseteq U$, authority permits U, and execution stays mediated under the same assumptions, covered admitted effects respect that authority. For a policy expressible as an allowed set B, the implication from $E \subseteq U$ and $U \subseteq B$ to $E \subseteq B$ is ordinary subset inclusion, not a new theorem. Establishing the premises is the hard part. History, aggregate budgets, and temporal constraints are not generally independent per-tuple membership tests. An observed trace also does not establish a complete upper bound on possible behavior.

Relevant faults include body/context/implementation drift, stale evidence, untrusted declarations, tuple recombination, changed authority, duplicate occurrences, and uncertain delivery. Arbitrary hidden SaaS behavior, covert channels, and unmediated effects outside the declared universe are not covered by the finite pilot.

3 Relation to Existing Interfaces

MCP. The selected `2025-11-25` schema includes read-only, destructive, idempotence, and open-world hints, with an explicit warning against trusting annotations from untrusted servers [S1]. It is not asserted to be the latest revision or the protocol actually negotiated in a run. A supported effect extension needs its own agreed semantics; there is no claim of an upstream-standard Action-Effect field. A capable MCP integration can already use a context-aware structured adapter, so hints-only comparison is insufficient.

AuthZEN and RAR. AuthZEN Authorization API 1.0 is Final, published 11 January 2026, and provides subject/action/resource/context decision exchange [S2]. Its successful decision is boolean with optional context. Review, unsupported semantics, and processing failure need explicit application-level mappings, not invented boolean values or automatic correctness credit for a failed PDP call.

RAR's structured `authorization_details` and separate objects already preserve distinct permission combinations; coupling is not novel merely because this draft also requires it. RFC 9396 section 6.1 explains that arbitrary details lack a universal comparison operation independent of type semantics [S3]. Requested details, granted details, and described effects are different. Token privilege restriction, audience checks, and actual resource authorization remain necessary [S8]. Reusable evidence does not cache permission.

OPA and Cedar. OPA accepts complete structured input. Its Data API can return HTTP 200 without a `result` for an undefined document, while built-in errors ordinarily evaluate to undefined unless the selected strict-error mechanism changes that behavior [S4, S5]. The required total-decision wrapper must validate outputs and declare its error configuration identically across arms.

Cedar provides principal/action/resource/context requests, entities, policies, and schema validation. Its documented default-deny, forbid-overrides-permit, and skip-on-error semantics return diagnostics; an error in one policy can coexist with an Allow from another [S6, S7]. That is not automatically a Cedar defect. A stricter wrapper or review route must be explicit and symmetric. Policy validation does not authenticate facts. Cedar is a future comparator, not an executed lane of the current OPA pilot.

Provenance, bytes, and provider behavior. PROV-DM structures provenance but does not establish factual truth [S9]. JCS defines canonical JSON, not authority or semantic correctness [S10]. The finite consumer's numerical subset must not be presented as full binary64/JCS support. GitHub documents notification effects and secondary rate limiting for comment creation [S11]. Recipients, delivery, webhooks, and other descendants remain unresolved under the partial profile. Reading a comment does not establish their absence. HTTP retry semantics likewise do not make an uncertain create safe to repeat [S12]; a local marker or MCP hint is not a provider idempotency key.

This is a focused compatibility review, not a complete scholarly novelty review of effect systems, abstract interpretation, capabilities, information-flow control, or agent governance. No superiority over an unintegrated system follows.

4 Bounded Runtime Pilot

The published Spec prerelease `v0.2.0-draft.1` resolves to `a9b86f4`, pinned in Bench's `spec-lock.json`. The measured Bench source is `9f638df`. GitHub Issue Comment Profile 1 is additive on unchanged core wire version `0.1.0-draft.1`; its released source is `profiles/github-issue-comment/README.md`. Native SDK input contains owner, repository, issue number, and body; the MCP lane uses `tools/call` for `add_issue_comment`. Lane-specific capture envelopes are not a replacement MCP protocol. Equivalent intents compare checked projections, not full description digests that include different native lanes.

Reports record MCP SDK `1.32.1`, Octokit `22.0.1`, OPA `1.21.2`, Node `22.18.0`, and Python `3.9.6`, with source and binary hashes. Offline evaluation uses local SDK/MCP-shaped captures and real OPA, not provider transport. The live MCP lane uses real stdio initialize/list/call; both live lanes use Octokit against GitHub.

The profile binds the exact issue and destination to body SHA-256 and UTF-8 byte count. Its finite input domain includes a nonempty Unicode-scalar body of at most 8192 bytes. Data classification is trusted configuration, not DLP; `github-comment-bridge/1` is a checked declaration, not attestation of provider code. Success is `accepted_partial/declared_partial`, still without authority. After applicable denials, the experiment policy requires `allow_partial_direct: true`; otherwise partial scope requires review.

The proposed arm uses producer, description, separate consumer, real OPA, and a final gate. The main baseline substitutes a complete structured adapter but shares policy, authenticated facts, gate, occurrence ledger, observation rules, and offered work (Figure 1). A lossy-field ablation is not the main competitor. Only approved synthetic operations on owned public test issues may be dispatched; a stub or replay cannot count as live evidence.

Before POST, reserve the occurrence and recheck current local experimental authority and request bindings. This is not an atomic transaction with GitHub or instantaneous provider revocation. Once delivery might have begun, retain the reservation and reconcile read-only rather than automatically repeat POST. A single-host ledger does not establish distributed exactly-once execution. Known response IDs must bind the exact readback. ID-less recovery requires a complete pre-dispatch ID list, a previously absent ID, and a unique exact body/actor match on the dedicated single-writer issue. Its evidence explicitly means new current content, not execution attribution. A concurrent same-account writer violates that assumption. Missing or ambiguous evidence stays unknown.

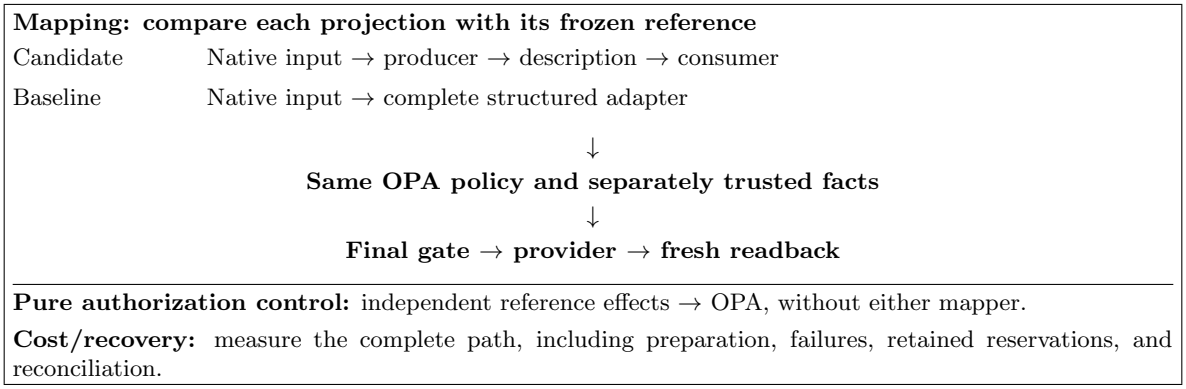


Figure 1: Measurement factorization. Offline scoring stops before dispatch; live execution also exercises the gate and provider. These stages have separate references. A correct-looking denial cannot conceal a wrong mapping, and a fault-induced stop is not semantic correctness.

5 Falsifiable Questions and Experimental Design

`PROTOCOL.md` governs the study; this paper does not silently change its corpora, labels, scoring, or independence requirements.

1. **Meaning preservation:** do both arms match reference coupled effects, binding state, and uncertainty across equivalent intents? Baseline parity supplies no added mapping benefit; extra loss or unsupported outcomes count against the proposed interchange.
2. **Diagnostic value:** can separate mapping, reference-input authorization, and integrated scores reveal policy-masked mapping defects? Report their joint correctness cells, with additional explicit failure cells. A complete baseline may expose the same distinctions.
3. **Unknowns and freshness:** do prewritten drift, authority-change, capability, and delivery cases preserve required limits? Fabricated known facts, stale acceptance, error-as-review, and permission reuse contradict the contract.
4. **Cost and utility:** does any demonstrated portability or protection benefit justify complete cost, withheld work, intervention, and recovery? Negative findings and a better baseline remain valid outcomes.

The base intent is the sampling unit; variants, repeats, languages, and lanes are not extra independent cases. This is an author-designed convenience pilot, not a customer-workflow population sample. Competence and tuning budgets must be matched in public development. An OPA evaluator cannot define the expected answers and then receive independent correctness credit for matching itself. Pure authorization uses separately authored reference effects and decisions.

Authoring and independence. Implementation, separate case authoring, and review were AI-assisted within one project. Literal references were authored without candidate-output inspection; implementation proceeded in parallel from the agreed public interface. The corrected custody record describes withholding cases until a *pre-execution/pre-result* freeze, not a pre-implementation freeze. Subsequent failures became exposed regressions. The editorial audit checks retained reports after execution, not independent reference authorship. No external evaluation or protected external cohort occurred.

The selected official OpenAPI commit is `755ee33b4a81d51f6f07a902cce26cbec3b822f2` [S13]. The pinned excerpt retains its MIT notice and supplies structural examples and notification wording, not independent governance labels. Official commit identity and local extraction hashes remain separate records. After a learned failure, that family becomes exposed development; hiding or hashing it does not restore unseen status.

Endpoint	Numerator	Denominator
False allow on block	Valid semantic allow in B	All B
False allow on review	Valid semantic allow in R	All R
False block	Valid semantic block in A	All A
Correct review	Valid semantic review in R	All R
Unnecessary review	Valid semantic review in A	All A
Legitimate work withheld	Any non-dispatch in A, including faults	All A
Forbidden dispatch	Dispatch contrary to the reference	All expected withhold

Table 1: Frozen-population endpoint definitions. A fault-induced stop is failure and possible utility loss, not correct semantic block/review. An all-block system cannot win through a single false-allow statistic.

Protected external evaluation additionally requires independently authored inputs and outcomes, separate sealed access, actual custodians, independent disputed-score governance, and disclosed conflicts. System exposure is not developer access. Hosted-provider logging can undermine secrecy and must be documented. Two languages, repositories, or AI-assisted sessions do not establish independent laboratories. Active protected traces and labels stay sealed until retirement and rights review.

6 Metrics and Analysis

Primary rates use their full frozen scheduled populations, including unsupported, missing, and failed output. Successful-only rates are supplementary. Pure malformed-wire cases occupy a predeclared separate track; no favorable post hoc exclusion is allowed.

Mapping and uncertainty. Report exact semantic match, coupled-tuple and per-field loss, correct scope, modality, and bindings, together with coverage and reference ambiguity. For reference-unknown slots, report correct unknown retention and unjustified known/none assertions; for reference-known slots, report avoidable unknowns. Missing output earns no unknown-retention credit. Capability/processing checks may correctly return unsupported without establishing successful effect mapping.

Decisions and admission. Let A, B, and R be the preassigned expected-allow, expected-block, and expected-review populations. Table 1 defines rates, not results. Retain full confusion tables including non-decisions for pure authorization and integrated admission. Record the gate and actual dispatch independently.

Failure, completion, and recovery. Separate invalid input/configuration/output, unsupported semantics, untrusted evidence, timeout, crash, missing output, and policy/provider/observer failures. Retain every attempt and link each permitted rerun; do not select the best run. Intentional non-dispatch is not attempted delivery. Provider outcomes can be confirmed, partial, unknown, or conflicting. Unknown-delivery rates use all attempted occurrences, also showing counts against scheduled work.

Original task completion uses all scheduled legitimate tasks; withholding, timeout, abandonment, and missing completion are non-completions. Report forbidden effects, duplicates, retained reservations, reconciliation, and human burden. Joint verified useful-and-safe completion needs the original goal, no forbidden effect in the declared universe/window, and sufficient observation. Unknown coverage cannot qualify. Direct-comment-only success must be called scoped verified completion, not safety of every descendant. Separate clean and challenged workloads and label conservative sensitivity bounds as bounds.

Complete cost and inference. Retain raw monotonic timings for preparation, parsing, mapping, serialization, validation, evidence checking, invalidation, authorization, final recheck, reservation, transport, provider response, readback, and reconciliation. Include end-to-end boundaries, cold/warm state, startup,

offered/rejected work, input size, load, resources, network, provider calls, and measured human or monetary cost. Do not sum overlapping timers or turn unmeasured cost into zero.

Use paired intent-level comparisons and predeclared family weighting, clustering variants and repeats. Small convenience samples warrant descriptive findings, not population coverage or unsupported tail-SLA claims. Report sample support for percentiles. Zero observed failures is not zero risk; correlated variants do not justify an independent-trial bound. Preserved utility or review burden needs justified preregistered non-inferiority criteria, not a nonsignificant difference. Publish negative results and analysis deviations.

7 Current Artifacts and Limitations

The legacy finite refund profile remains separate; its original-corpus rerun matched 102/102 expected outputs per consumer with no adapter faults. Released opt-in `offline-consumer-v1.1` clarifies startup and compound-error precedence without rewriting frozen v1. These are not new runtime workloads.

Linux/macOS CI succeeded at the measured commit. Reported totals are 64 Spec JavaScript, 11 legacy Bench JavaScript, 115 Spec Python, 333 runtime JavaScript, and 50 runtime Python methods: 573 methods, not benchmark cases.

The scope remains one synthetic public operation, not arbitrary tools, autonomous-model behavior, or customer workflows. Notifications, recipients, delivery, webhooks, and other descendants stay unknown. The host, configured principal/class, policy files, provider adapter, and same-host ledger are trusted. The bridge identity is declared, not attested SaaS code.

Snapshots across awaits and exact response/readback ID binding close specific reviewed defects, not every concurrency or revocation race. Local policy and remote mutation remain non-atomic. ID-less recovery is conditional on complete preflight and a dedicated single writer; new exact content is not execution attribution. A same-account concurrent writer remains outside that guarantee. Hashes and replay do not authenticate an operator or establish historical fsync.

Internal review reports eight gate regressions corrected before the measured source, involving stale/edited content, ID binding, mutable configuration, and policy freshness. This is development history, not another cohort. Shared contracts and policy permit correlated mistakes. External independently authored evaluations remain zero. Full descendant coverage, independent implementations, realistic load, operational effort, and deployment readiness remain unestablished.

8 Results

8.1 Sources and Denominators

The offline and live reports use the linked Bench and Spec commits above and the same 49-entry source inventory; each entry matches its frozen Git blob in the editorial audit. The recorded Bench checkout is dirty, including reports and documentation, not represented as clean. Exact filenames, hashes and commands are in `measurement-contract.md` and `sources.json`.

There are 55 authored cases in 22 base-intent groups, with 53 SDK and 55 MCP inputs: 108 lane instances per strategy and 216 strategy rows. Two omitted SDK renderings are inapplicable MCP-specific inputs, not failures. Accepted input comprises 26 authored cases in 13 groups, rendered in both lanes: 52 rows per strategy. These related observations do not support independent-trial or population-accuracy claims.

8.2 Offline Mapping and Authorization

Check, per strategy	Protocol	Structured
Exact status, reason and supported projection	108/108	108/108
Accepted partial mappings	52	52
Invalid / untrusted / unsupported	51 / 4 / 1	51 / 4 / 1
Exact eligible decisions	52/52	52/52
Allow / deny / review	22 / 28 / 2	22 / 28 / 2
Authorization not evaluated	56	56
False allows on deny/review	0/30	0/30
Expected allows not allowed	0/22	0/22
Real masked errors / operational failures	0 / 0	0 / 0

Pure authorization matched 26/26 separately authored reference decisions: 11 allow, 14 deny, and one review. These controls share the authored cases; they are not an additional independent cohort. Mapping nonacceptance is not credited as policy denial.

A deliberately lossy diagnostic replaced 26 reference body digests with zeros: mapping matched 0/26 while policy decisions matched 26/26. This illustrates why decision accuracy cannot substitute for mapping checks. It is not a competitor weakness, empirical attack rate, or naturally observed error in either complete strategy.

8.3 Controlled Live Outcomes

The run on issue #1 lasted from 04:24:32.195Z to 04:26:11.245Z on 19 October 2026. Eight scripted scenarios crossed with two strategies and two lanes yield 32 scheduled checks, not 32 independent incidents or successful creation tasks.

Observed category, per strategy	Protocol	Structured
Planned outcome/reason checks	16/16	16/16
Ordinary immediate confirmations	4	4
Lost ACK: unknown then read-only confirmed	2	2
Duplicate occurrences suppressed	2	2
Withheld: deny / review / invalid-policy failure	4 / 2 / 2	4 / 2 / 2

The issue started empty and ended with 12 run-scoped new comments: eight normal writes and four controlled lost-ACK writes. Duplicate occurrences added none. The four initial unknowns are 4/12 new-create attempts and 4/32 scheduled checks by design, not an outage estimate. Each has a `successful_post_ack_discarded` witness and read-only recovery with `automatic_resend:false`. The injected loss follows a real successful POST response, not a naturally occurring outage. Recovery establishes new current content, not general execution attribution.

Four invalid-policy rows are evaluation failures, correctly withheld, not semantic denials. Thus 32/32 planned checks does not mean every row succeeded operationally. Direct-content confirmation leaves downstream effects unknown.

8.4 Cost Without a Speed Ranking

Median elapsed time, ms	Protocol	Structured	n/path
Accepted offline input	50.391	48.468	52
All offline input	0.042	44.507	108
Ordinary live completion	3110.848	3161.205	4

Accepted-input median cost is 1.923 ms, or **3.97% higher**, for the protocol. This is a ratio of marginal medians, not a median paired percentage or significance claim. Accepted includes deny/review: it is not provider-task latency. The all-input median crosses a different boundary: 56 protocol rows stop in JavaScript without Python startup, while the direct adapter starts Python. It is not comparable useful-task speedup.

Accepted offline cost includes cold Python communication/checking and real OPA startup. Live cost also includes identity lookup, preflight, ledger, POST and readback, plus cold MCP transport where applicable. The entire run took 99.049 seconds, including one-second pacing waits excluded from row timers. Initial live rows overlapped same-host read-only offline replay; the host was not isolated. Four normal observations per path support neither speed ranking nor SLA. The host reports Darwin arm64, Apple M4 Pro, 14 logical CPUs and 24 GiB memory; Toronto is editorial metadata, not geolocation.

8.5 Retained Failure and Replay

The first failed offline report is retained. Its scorer reported 52 apparent masked errors per strategy from null-prototype versus ordinary-object comparison. A JSON-value audit finds 108 exact protocol and 100 exact structured mappings; eight structured numeric/Unicode error-classification mismatches remain. These are not 52 real policy-masked defects. All 216 input/context/ policy and expected records are unchanged in the final report. Repairs changed code and scoring, not labels. The complete initial executable tree was not archived; this is retained evidence, not exact old-source reproduction.

Offline replay passed 216 strategy rows plus 52 reference/diagnostic records. Live replay passed 32 rows and matched all 12 unique comments using anonymous GETs at its recorded check time. The attempt journal has one begin, 32 scheduled, 32 result, and one end record consistent with the live report. Replay verifies recorded consistency, calculations, and current content, not operator honesty, historical latency, fsync, or complete causality. Later unavailable, changed, or rate-limited content cannot be treated as a match. The full artifact map is in `measurement-contract.md`.

9 Adoption, Rights, and Publication Boundary

Adoption is staged and interface-only: bounded offline implementation, non-dispatching comparison, then a separately authorized owned-resource test. No OSuite endpoint, account, module, purchase, or code is required. The adoption quickstart documents implemented installation, offline, replay, and authorized-live interfaces with automatic read-only lost-ACK recovery. Supplied unit tests and source inspection do not establish completed measurements, customer deployment readiness, or a reason to remove existing controls.

The public RFC asks for concrete tuple-loss, drift, unknown/failure, freshness, recovery, and fair-baseline tests. Protected inputs and outcomes must not be posted publicly. Unrelated protected collaboration data is excluded. Outreach copy is unsent material, not evidence of participation or endorsement.

The repositories retain MIT for selected original contributions; this document does not change that license. Third-party sources and outside submissions retain their applicable rights and need appropriate authority for incorporation. MIT publication is not patent/legal clearance, a separate patent covenant, certification, standards-body approval, or another party's authorship consent. This institutional working paper is for public review, not an automatic formal submission. Formal submission or an individual-author byline requires separate confirmation. No external coauthor, endorsement, or custodial appointment is implied; outreach copy remains unsent.

10 Conclusion

Action-Effect is useful only if its interchange preserves supported meaning, exposes limits, and enables controlled useful work at a justified cost relative to competent existing integrations. The proposed factorization makes masking, failures, and parity visible. The completed bounded study found equal mapping and authorization checks, equal planned live outcomes, and higher accepted-input offline median cost for the protocol. It supplies an inspectable interchange and empirical method, not demonstrated superiority over a complete structured adapter. Independent evaluation and broader operational evidence remain open.

References

[S1] Model Context Protocol. *2025-11-25 schema*, selected source commit `c4c367f9f58296a7053f5c78a52fd02bfbb56a49`; tool annotations. https://github.com/modelcontextprotocol/modelcontextprotocol/blob/c4c367f9f58296a7053f5c78a52fd02bfbb56a49/schema/2025-11-25/schema.ts.

[S2] OpenID Foundation. *Authorization API 1.0*. Final, 11 January 2026, sections 5–7. https://openid.net/specs/authorization-api-1_0.html.

[S3] T. Lodderstedt, J. Richer, and B. Campbell. *OAuth 2.0 Rich Authorization Requests*. RFC 9396, May 2023, sections 2.2, 6.1, 7, and 9. https://www.rfc-editor.org/rfc/rfc9396.txt.

[S4] Open Policy Agent. *REST API documentation*, Data API. Documentation commit `7bcf86187363b525802096f4668c2db1e058a831`, not an engine-run pin. https://github.com/open-policy-agent/opa/blob/7bcf86187363b525802096f4668c2db1e058a831/docs/docs/rest-api.md.

[S5] Open Policy Agent. *Policy Language documentation*, objects, defaults, and built-in errors. Same documentation commit as S4. https://github.com/open-policy-agent/opa/blob/7bcf86187363b525802096f4668c2db1e058a831/docs/docs/policy-language.md.

[S6] Cedar. *Authorization documentation*. Documentation commit `3a37098eba6e02e716949cfc92b889b38db25602`; no Cedar execution claim. https://github.com/cedar-policy/cedar-docs/blob/3a37098eba6e02e716949cfc92b889b38db25602/docs/collections/_auth/authorization.md.

[S7] Cedar. *Policy validation against schema*. Same documentation commit as S6. https://github.com/cedar-policy/cedar-docs/blob/3a37098eba6e02e716949cfc92b889b38db25602/docs/collections/_policies/validation.md.

[S8] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett. *Best Current Practice for OAuth 2.0 Security*. RFC 9700, January 2025, section 2.3. https://www.rfc-editor.org/rfc/rfc9700.txt.

[S9] W3C. *PROV-DM: The PROV Data Model*. Recommendation, 30 April 2013. https://www.w3.org/TR/2013/REC-prov-dm-20130430/.

[S10] A. Rundgren, B. Jordan, and S. Erdtman. *JSON Canonicalization Scheme (JCS)*. RFC 8785, Informational, June 2020. https://www.rfc-editor.org/rfc/rfc8785.txt.

[S11] GitHub. *REST API endpoints for issue comments*. Selected documentation API version `2022-11-28`; create/get comment and notification notice. https://docs.github.com/en/rest/issues/comments?apiVersion=2022-11-28.

[S12] R. Fielding, M. Nottingham, and J. Reschke. *HTTP Semantics*. RFC 9110, June 2022, section 9.2.2. https://www.rfc-editor.org/rfc/rfc9110.txt.

[S13] GitHub. *Official REST API description source commit* `755ee33b4a81d51f6f07a902cce26cbec3b822f2`. Commit identity verified; not the extracted operation bytes or independent governance labels. https://api.github.com/repos/github/rest-api-description/commits/755ee33b4a81d51f6f07a902cce26cbec3b822f2.

Source keys correspond to `sources.json`, which records exact verified retrieval URLs, versions, UTC timestamps, body hashes, scope limits, and an incomplete retrieval. Raw third-party source captures are not redistributed. This file embeds its bibliography and factorization figure; it does not require those companion files to compile.